

VaultX Cyber Tech AI Internship

Week 5 Report

RAG + Tool-Calling Agent (Slack Delivery)

Intern	Aziz ur Rehman
Program	BS Artificial Intelligence
Internship	VaultX Cyber Tech AI
Week	Week 5 (Self-Directed Project)
Repository	chaziz-ai/vaultx-ai-internship (branch: main)

1. Project Overview

No official task document was provided for Week 5. The instruction given was to build a project demonstrating the skills learned in Weeks 1–4. Based on this, the following self-directed project was scoped and built:

Project: A RAG + Tool-Calling Agent that answers questions through a chat interface by automatically deciding, for each question, which of three tools to use:

- **search_documents** — retrieves answers from company PDF documents using the Week 4 ChromaDB RAG pipeline
- **search_web** — retrieves live/current information from the web
- **calculator** — performs mathematical calculations

This introduces **tool / function calling**, a new concept for Week 5: instead of the model only generating text, it can now request that a specific Python function be executed, receive the result, and use it to produce a grounded final answer. This is a genuine upgrade over Week 4, where retrieval was the only capability the system had.

Why this matters

A plain LLM has no access to real-time data, cannot reliably perform exact arithmetic, and cannot search an organization's private documents unless that content is manually placed in its context. Tool calling gives the model a way to decide, per query, which external capability it needs and to call it — closer to how a real AI agent operates.

2. Tech Stack & Architecture

Tools and Technologies Used

- Python + OpenAI API (gpt-4o-mini) — carried over from Weeks 1–4
- ChromaDB + embeddings — reused RAG pipeline from Week 4
- FastAPI — wraps the agent as an HTTP endpoint (/chat)
- n8n (self-hosted, Docker) — carried over from Week 3, used to connect Slack to the agent
- Slack — used as the delivery/chat interface for this project
- ngrok — exposes the local FastAPI agent to n8n/webhooks

Agent Architecture

Tools are defined using the OpenAI tool-calling "menu card" pattern: each tool is described to the model with a name, description, and expected parameters (JSON schema). When a user question arrives, the model inspects the available tools and decides whether a tool call is needed, and if so, which one and with what arguments. The requested tool is then executed locally in Python, and its result is sent back to the model (as a role: "tool" message) so it can generate the final, grounded answer.

Data Flow (Slack → Agent → Slack)

Slack message → n8n Slack Trigger → If node (filters bot's own messages to avoid loops) → Get rows (pulls last 5 messages for memory) → Code node (formats history) → HTTP Request (calls the FastAPI agent) → Send message (reply back to Slack) → Append row (logs the interaction to Google Sheets). A parallel error branch sends a friendly fallback message and logs the error separately if the HTTP Request fails.

3. Implementation Summary

Files / Components Built

- **test_tool_calling.py** — standalone script used to build and verify the tool-calling loop in the terminal before wiring it into the API
- **api.py** — FastAPI wrapper exposing a /chat endpoint, tracks which tool was used per request, supports multi-language input (including Roman Urdu)
- **n8n workflow (Week05_Agent_Slack_Query)** — fully working: conversation memory (last 5 messages), error handling with fallback + logging, Google Sheets logging (Timestamp, Question, Answer, Tool Used)

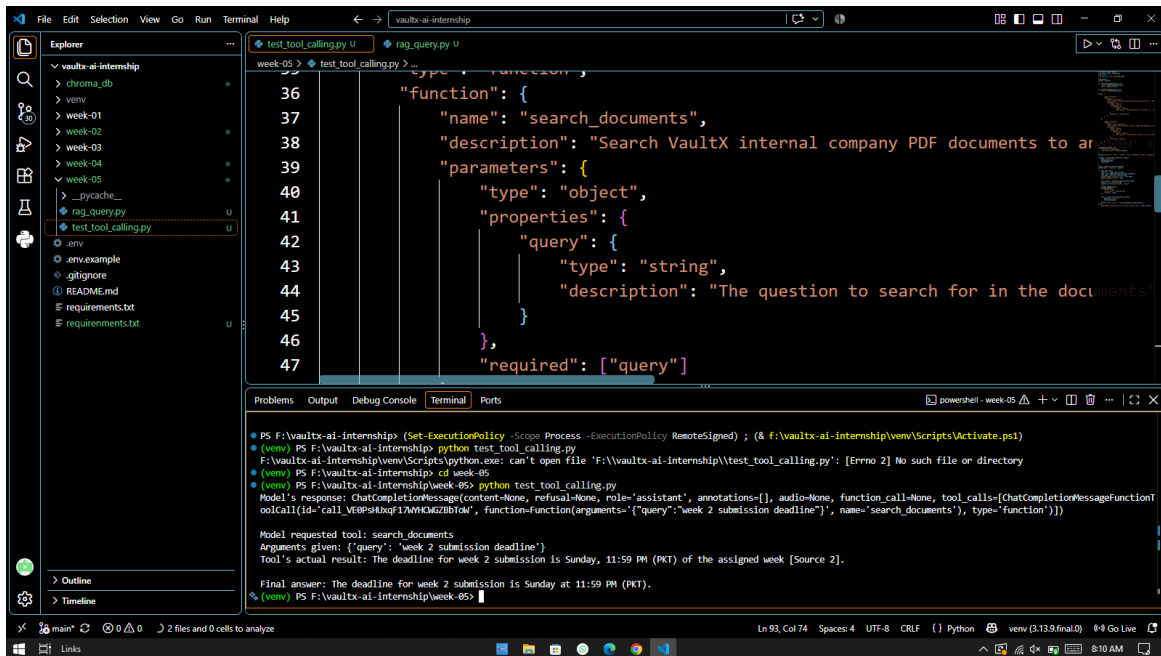
Reused from Previous Weeks

The RAG pipeline (chunking, ChromaDB collection, retrieval + grounded generation) was reused directly from Week 4 rather than rebuilt, and wrapped as the `search_documents` tool. The n8n error-handling and logging patterns established in Week 3 were also reused and adapted for Slack.

4. Testing & Evidence

4.1 search_documents Tool — Verified via Terminal

Before integrating into the API/Slack flow, the tool-calling loop was tested directly in the terminal. The model correctly identified that a question about an internal deadline required the `search_documents` tool, called it with an appropriate query, and used the retrieved result to generate an accurate final answer.



```
File Edit Selection View Go Run Terminal Help
vaultx-ai-internship
test_tool_calling.py U rag_query.py U
week-05 > test_tool_calling.py >
36
37
38
39
40
41
42
43
44
45
46
47
"function": {
  "name": "search_documents",
  "description": "Search VaultX internal company PDF documents to ar
  "parameters": {
    "type": "object",
    "properties": {
      "query": {
        "type": "string",
        "description": "The question to search for in the documents"
      }
    }
  },
  "required": ["query"]
}

PS F:\vaultx-ai-internship> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& f:\vaultx-ai-internship\venv\Scripts\Activate.ps1)
PS F:\vaultx-ai-internship> python test_tool_calling.py
F:\vaultx-ai-internship\venv\Scripts\python.exe: can't open file 'F:\vaultx-ai-internship\test_tool_calling.py': [Errno 2] No such file or directory
PS F:\vaultx-ai-internship> cd week-05
PS F:\vaultx-ai-internship\week-05> python test_tool_calling.py
Model's response: ChatCompletionMessage(content=None, refusal=None, role='assistant', annotations=[], audio=None, function_call=None, tool_calls=[ChatCompletionMessageFunctionToolCall(id='call_V08Pshqp17MMkMzBb1dW', function=Function(arguments={'query': 'week 2 submission deadline'}, name='search_documents'), type='function')])
Model requested tool: search_documents
Arguments given: {'query': 'week 2 submission deadline'}
Tool's actual result: The deadline for week 2 submission is Sunday, 11:59 PM (PKT) of the assigned week [Source 2].
Final answer: The deadline for week 2 submission is Sunday at 11:59 PM (PKT).
PS F:\vaultx-ai-internship\week-05>
```

Fig 1: Terminal output showing the model requesting the search_documents tool, the retrieved result, and the final grounded answer.

4.2 Calculator Tool — Proven More Accurate Than Plain ChatGPT

The same arithmetic question (8473×692) was asked to plain ChatGPT and to the Slack agent. Plain ChatGPT produced an incorrect result, while the agent's calculator tool (which executes real Python arithmetic rather than the model guessing) produced the correct result. This is direct evidence that tool calling improves reliability over relying on the LLM alone for computation.

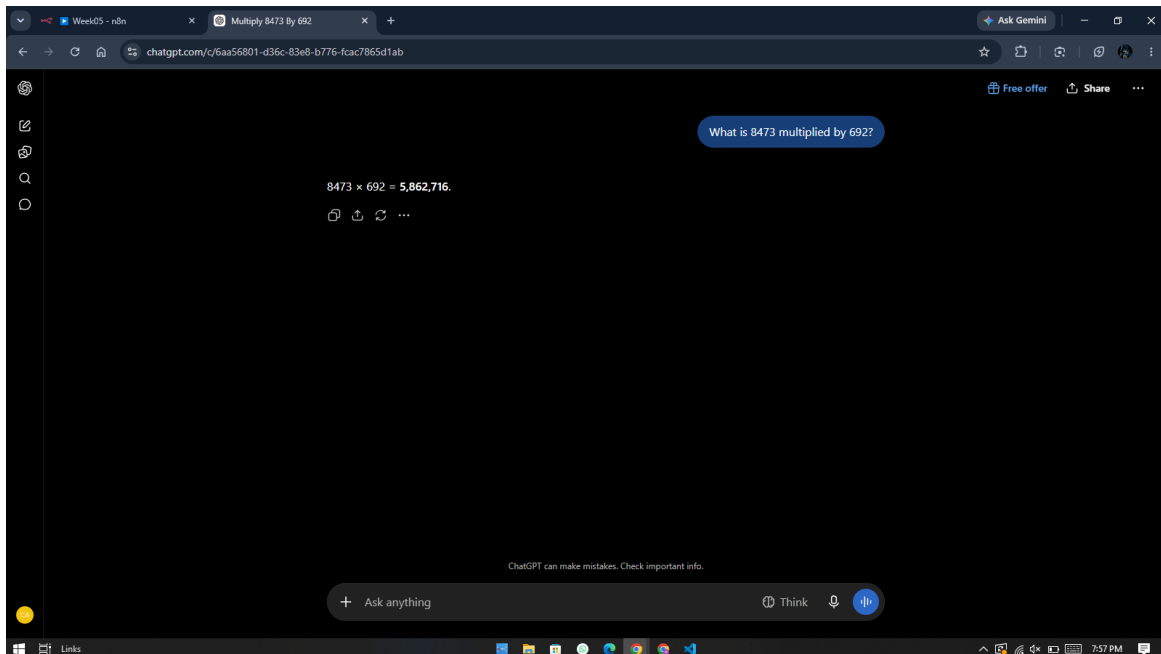


Fig 2: Plain ChatGPT answers $8473 \times 692 = 5,862,716$ — this is INCORRECT.

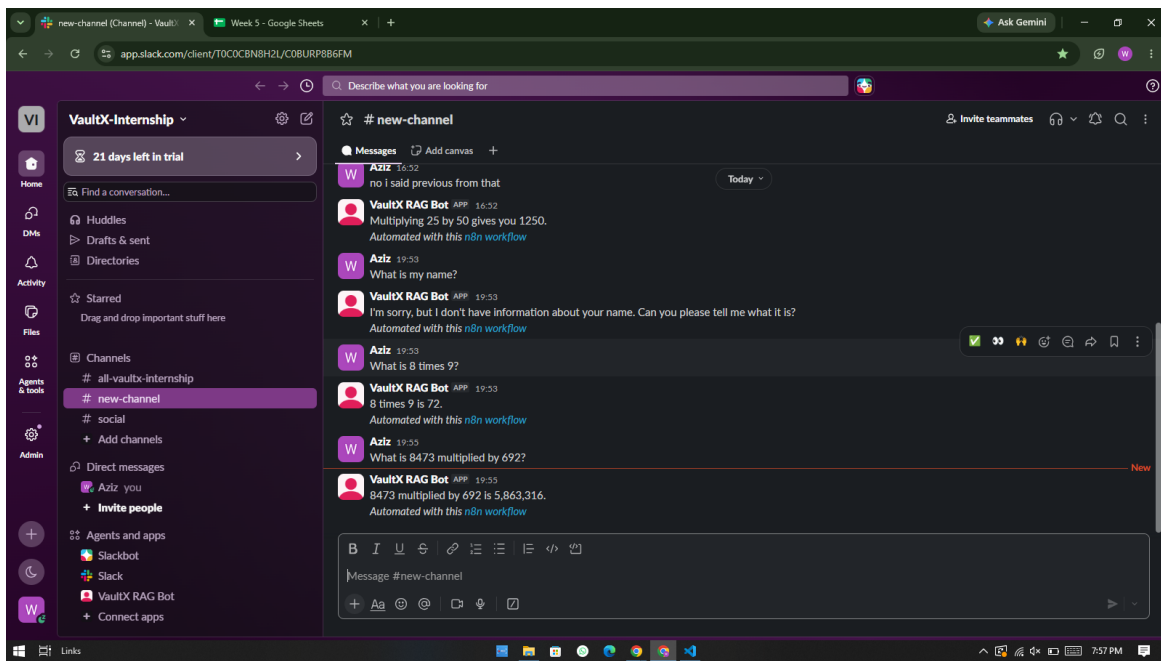


Fig 3: The Slack agent answers the same question via the calculator tool = 5,863,316 — this is the CORRECT result (verified against actual multiplication).

4.3 Multi-Step / Context Questions — Memory Behaviour

The agent was tested with a chain of follow-up questions relying on conversation memory (e.g. "multiply that by 2", then "multiply this by previous answer"). The agent used the last 5 logged messages to attempt to resolve "previous answer", but interpretation of ambiguous references was not always exactly what the user intended — this is documented as a known limitation in Section 5.

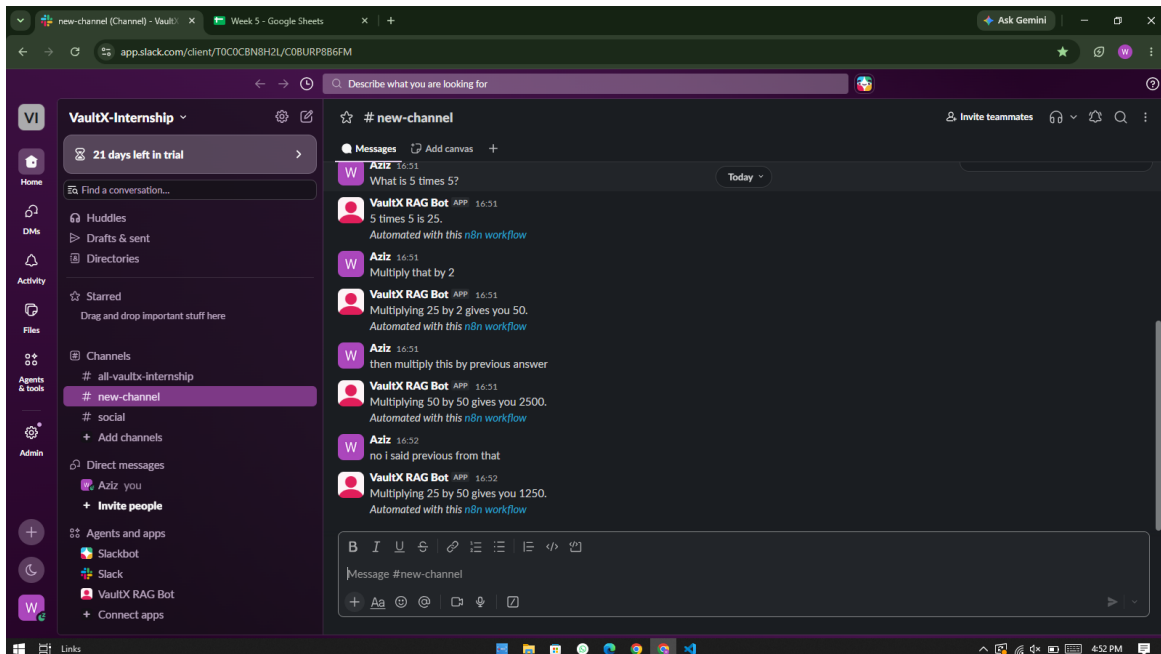


Fig 4: Chained calculator questions using conversational memory in Slack.

4.4 Complex Multi-Step Question — Imprecise but Plausible Answer

A more complex question combining a stored deadline with date arithmetic was tested. The agent gave a plausible-sounding answer, but it was not fully precise — illustrating a limitation of relying on the LLM to reason over multi-step date logic rather than delegating it to a dedicated function.

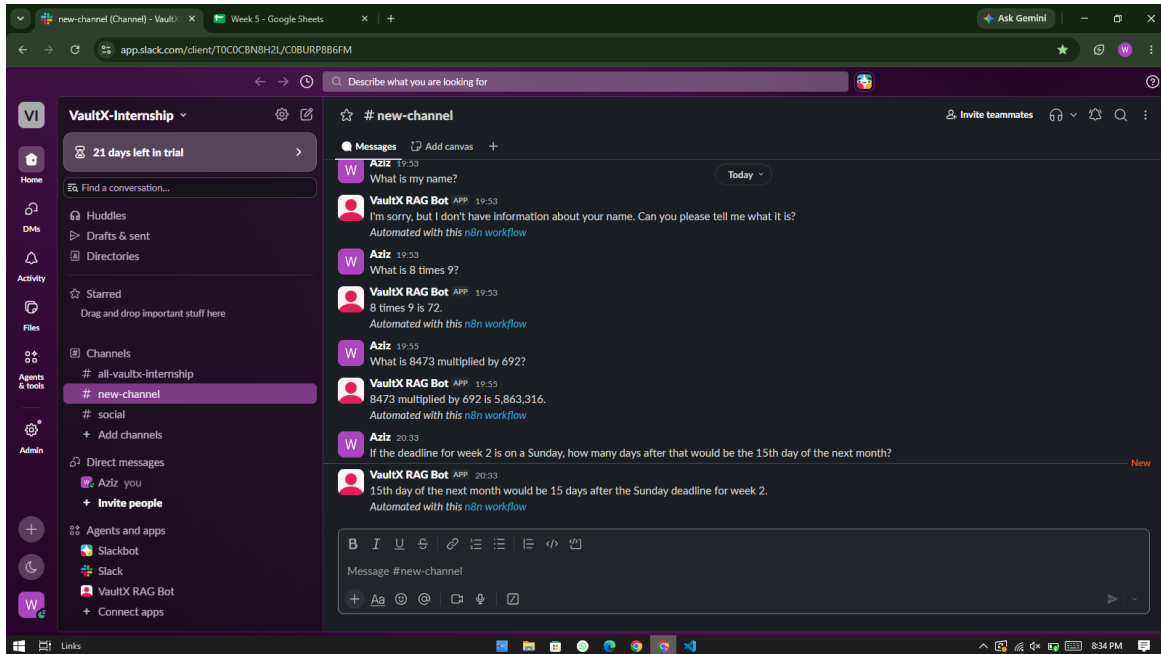


Fig 5: Agent handling 'What is my name?', simple calculator queries, and a compound date-reasoning question in the same conversation.

5. Refinements After Initial Testing

While re-testing the Slack integration ahead of final submission, two additional issues were discovered and fixed in the agent's system prompt (api.py). Both are recorded here as part of the development process, alongside the automation improvement made to the n8n workflow at the same time.

5.1 Language Was Not Following the Latest Question

The agent was found to reply in Roman Urdu even when the user's most recent question was in English. The cause was that the conversation history sent to the model (last 5 messages, used for memory) was mostly in Roman Urdu, and the model's language choice was being pulled toward that history rather than the newest question — a known LLM tendency to weight earlier context heavily. **Fix:** the language instruction was moved to a separate system message placed *after* the user's question in the message list (rather than only before it), explicitly telling the model to identify the newest question and ignore the language of earlier history when choosing its reply language. After this change, English questions were correctly answered in English and Roman Urdu questions in Roman Urdu, independent of the conversation history's language.

5.2 Date Questions Were Answered from Stale Training Data

Asking the agent "What is today's date?" initially returned an incorrect, outdated date, indicating the model was guessing from its own training data instead of invoking the `web_search` tool. **Fix:** the system prompt was updated to explicitly state that the model does not reliably know the current date/time and must use the `web_search` tool for any such question rather than answering from memory. After this change, the agent correctly retrieved and returned the actual current date via web search.

5.3 n8n Workflow: Added a Rate Limit Delay Node

A `Wait` node ("Rate Limit Delay", 3 seconds) was added between the history-formatting `Code` node and the `HTTP Request` node in the Slack workflow. This introduces a small, deliberate pause before each call to the FastAPI agent, which is a simple safeguard against sending requests too rapidly if a user sends several messages in quick succession.

6. Known Limitations

Rather than over-engineering fixes for every edge case, the following known limitations are being documented explicitly, consistent with the approach used in the Week 4 report for chunking-boundary issues:

- **Calculator uses Python's `eval()`:** chosen for simplicity within the project timeframe. A production system should use a safer expression evaluator such as `numexpr` or a restricted parser to avoid arbitrary code execution risk. This tradeoff was a deliberate, acknowledged decision.
- **Ambiguous multi-step references:** when a user refers to "the previous answer" across several turns, the agent does not always resolve the reference exactly as the user intends (see Section 4.3).
- **Multi-step date/deadline reasoning:** compound questions combining stored facts with date arithmetic can produce plausible but imprecise answers (see Section 4.4), since this reasoning is left to the LLM rather than a dedicated date-calculation tool.
- **WhatsApp integration attempted but blocked:** a Meta WhatsApp Business API test number and app were set up, and an n8n WhatsApp Trigger node was configured with valid, tested OAuth credentials. However, publishing the workflow consistently failed with a "Bad Request – Invalid parameter" error from the WhatsApp Trigger node, which could not be resolved within the project's time budget. As a result, the delivery channel was finalized as Slack, and WhatsApp integration is documented here as an attempted-but-blocked stretch goal rather than a completed deliverable.

7. Conclusion / Learnings

This project extended the Week 4 RAG system into a tool-calling agent capable of choosing between document retrieval, live web search, and calculation based on the nature of each question. The clearest practical takeaway was the calculator accuracy test: it directly demonstrated why giving an LLM real tools (rather than letting it guess) produces more reliable results. Conversation memory and error handling, both introduced in earlier weeks, were successfully carried forward and adapted to a new delivery channel (Slack). The WhatsApp integration attempt, while unsuccessful, was a useful exercise in diagnosing third-party API/webhook issues and in making a pragmatic decision to finalize on a working channel rather

than let one blocked integration stall the rest of the deliverable.